

# A SYSTEMIC APPROACH TO MAXIMIZE HETEROGENEOUS SYSTEM PERFORMANCE

---

A Dissertation  
Presented to  
the Graduate School of  
Clemson University

---

In Partial Fulfillment  
of the Requirements for the Degree  
Doctor of Philosophy  
Computer Science

---

by  
Thomas Lenzy Randall  
May 2025

---

Accepted by:  
Dr. Rong Ge, Committee Chair  
Dr. Kai Liu  
Dr. Feng Luo  
Dr. Prasanna Balaprakash  
Dr. Xingfu Wu

# Plain Language Abstract

My research focuses on maximizing the performance of some of the world's biggest applications running on the world's biggest computers. These machines are so large and complicated that it takes dedicated efforts to get the biggest bang for your buck (and these machines aren't cheap, so we really want it!). The research in this dissertation is composed of three connected parts: How do we measure the opportunities provided to us by new hardware? How do we design our applications to reduce the need to re-design them as hardware improves? How do we configure our hardware and software to work together effectively?

I provide deep insights into the opportunities that we can leverage right now to prepare us for future developments. This means that we can extend the lifetime of our efforts and solve new and bigger problems instead of just struggling to keep up with the problems that we're already working on. When applied in the United States Department of Energy, this means that my research helps other scientists do their work faster and with less taxpayer dollars footing the bill for energy and computer maintenance. My research also supports smaller organizations who many not have the expertise, manpower or resources to optimize every tiny detail. Because my work provides a lower bound on costs and delivers results quickly even when there isn't a lot of data to work with, I'm making it easier for everyone to get the most of new computing technologies.

# Abstract

Continuous increases in high performance computing (HPC) throughput have served as catalysts for industry and scientific advancement in countless manners that have fundamentally shaped our modern world. Our demands on compute resources continue to scale, but the limitations of Ahmdal’s law and Dennard scaling have proven increasingly difficult to overcome when approached solely through hardware or software design. Furthermore, even on the most advanced supercomputers, many HPC applications fail to utilize the collective system’s performance.

However, the resurgence of AI in industry has promoted an explosion of hardware and software codesign that have fueled massive improvements in GPU design and novel ASICs. These performance improvements are maximized on a broad variety of heterogeneous systems by specially tuning applications. Mimicking these developments across the whole of computing will require similarly holistic approaches combining specialty hardware, software that caters its design to the greatest strength of hardware, and fine-tuning on individual systems to truly maximize performance.

We use three distinct perspectives to holistically address scalable system performance. We analyze the impacts of liquid immersion cooling technologies on sustained application performance and energy efficiency. Next, we present a case study where intentional algorithmic redesign for GPU acceleration permits robust performance improvements that endure through multiple generations of hardware. We find that memory latency forms a primary bottleneck for GPU-accelerated performance and demonstrate how algorithm-specific optimizations can greatly improve performance over multiple architecture generations. Finally, we tie these concepts together in the form of performance optimization techniques that respect both software- and hardware-based performance constraints. We improve the re-usability of performance insights with novel transfer learning techniques that make the cost of performance optimization more predictable and more successful in the short-term. Our insights demonstrate the necessity of systemic approaches for performance tuning in HPC.

# Table of Contents

|                                                                            |            |
|----------------------------------------------------------------------------|------------|
| <b>Title Page</b> . . . . .                                                | <b>i</b>   |
| <b>Plain Language Abstract</b> . . . . .                                   | <b>ii</b>  |
| <b>Abstract</b> . . . . .                                                  | <b>iii</b> |
| <b>List of Tables</b> . . . . .                                            | <b>v</b>   |
| <b>List of Figures</b> . . . . .                                           | <b>vi</b>  |
| <b>1 Introduction</b> . . . . .                                            | <b>1</b>   |
| <b>2 Background</b> . . . . .                                              | <b>3</b>   |
| 2.1 Single-Phase Liquid Immersion Cooling (SPLC) Technologies . . . . .    | 3          |
| 2.2 The Word2Vec Algorithm and Its Optimization . . . . .                  | 4          |
| 2.3 Performance Autotuning . . . . .                                       | 4          |
| <b>3 Impacts of Immersion Cooling on Application Performance</b> . . . . . | <b>5</b>   |
| 3.1 Introduction . . . . .                                                 | 5          |
| 3.2 Related Work . . . . .                                                 | 7          |
| 3.3 Research Questions . . . . .                                           | 7          |
| 3.4 Experiment Design . . . . .                                            | 9          |
| 3.5 Results . . . . .                                                      | 16         |
| 3.6 Conclusions . . . . .                                                  | 21         |
| <b>4 PROPOSAL DEEPPGREEN</b> . . . . .                                     | <b>23</b>  |
| <b>5 Summary Discussion and Future Work</b> . . . . .                      | <b>24</b>  |
| 5.1 Research Questions to be Answered . . . . .                            | 25         |
| 5.2 Recommendations for Further Research . . . . .                         | 26         |
| <b>Bibliography</b> . . . . .                                              | <b>27</b>  |

# List of Tables

|     |                                                                     |    |
|-----|---------------------------------------------------------------------|----|
| 3.1 | Immersed Server Hardware . . . . .                                  | 11 |
| 3.2 | Air-Cooled Server Hardware . . . . .                                | 11 |
| 3.3 | Selected applications and classifications . . . . .                 | 12 |
| 3.4 | NVIDIA GPU metrics recorded using NVML . . . . .                    | 14 |
| 3.5 | Submer SmartPod metrics recorded from vendor-provided API . . . . . | 15 |
| 3.6 | Thermal Behaviors by Hardware Component . . . . .                   | 17 |

# List of Figures

|     |                                                                                                                                                                                |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | SPLIC tank arrangement used in our experiments. . . . .                                                                                                                        | 10 |
| 3.2 | Thermal Behavior of DGEMM Without Chilled Water. DGEMM runs on GPU 1. . .                                                                                                      | 18 |
| 3.3 | Thermal Behavior of DGEMM With Chilled Water. DGEMM runs on GPU 1. . . .                                                                                                       | 18 |
| 3.4 | Temperature at start of idle pump cycle vs amount of heat removed. . . . .                                                                                                     | 19 |
| 3.5 | Time intervals required by applications to accumulate heat. The smaller the interval,<br>the faster the application execution increases coolant temperature by 1 °Celsius. . . | 20 |

# Chapter 1

## Introduction

TLR: Motivaton -> RQs & Contrbutions -> Overview

Building and operating supercomputers are massive undertakings that ultimately fall short of expectations wherever the hardware and software are not well-aligned.

---

**In order for systemic solutions to be effective, they must be designed against the actual constraints of importance that limit performance.**

---

While decades of research have already dedicated significant efforts to improve system performance, the so-called “golden age of computer architectures” [1] has dramatically accelerated both the raw throughput of data computation and the complications of efforts to efficiently utilize hardware. While CPU hierarchies are well-understood and GPU systems have gained at least some footing, the rapid evolution of system design has left many systems regularly underperforming their theoretical peaks. Continual large-scale government and industry investments in specialized hardware for artificial intelligence, machine learning, cloud infrastructure and datacenter technologies have vastly outpaced our ability to maximize the utilization of these systems.

Many advancements in specialized hardware have been accompanied by increasing power consumption and thermal waste, raising legitimate concerns about the energy efficiency and environmental impacts of current and future large-scale computing. We also observe many applications failing to improve key metric performance at the rate of new hardware acquisition, suggesting unfortunate mismatches between software design and hardware implementation that are inefficiently addressed by newer technologies. Finally, we observe the largest gap in system performance is cre-

ated by the immense efforts needed versus finite resources allocated for performance tuning. Even when efforts are made to facilitate the usage of new hardware with existing software, performance improvements remain lackluster compared to the cost of acquisition.

In this work, we consider the impacts of hardware upon software and vice-versa from three distinct perspectives: hardware opportunity, software leverage and automatic performance optimization. As the number of hardware and software combinations are exponentially growing, we demonstrate how these perspectives are applied as a systemic approach to maximize heterogeneous system performance both now and in the future.

First, we consider the performance opportunities provided to systems based on novel hardware via a case study. We analyze the impacts of a Single-Phase Liquid Immersion Cooling (SPLIC) technology on cooling to determine how current and future applications may adapt to different physical constraints provided by this environment.

Second, we consider the complexity of designing software to properly exploit available performance across many generations of accelerator hardware through the lens of GPU implementations of the Word2Vec algorithm. We optimize Word2Vec performance on GPU accelerators to permit automatic performance improvements from successive hardware generations that have been neglected in prior work, remedying a history of poor GPU utilization that was previously outmatched by CPU implementations.

Third, we unite these perspectives of hardware- and software- based optimization through automatic empirical performance tuning, also known as performance autotuning. Once performance opportunities are known, the process of tuning available interfaces for maximal impact is nontrivial. Existing performance autotuning has limited capability to exploit prior knowledge and is forced to waste some empirical samples when tuning related problems. We identify a novel transfer tuning technique that immediately leverages prior knowledge to access near-optimal areas of tuning search spaces, allowing practitioners to expand the scope and utilization of autotuning to applications at low and predictable cost.

**TLR: NEED FINAL PARAGRAPH HERE**



## Chapter 2

# Background

### 2.1 Single-Phase Liquid Immersion Cooling (SPLC) Technologies

One of the major ways to iteratively improve performance through hardware is to increase the density of computing components. However, denser circuitry increases the power density of the components and reduces cooling effectiveness for components such as those deeper within 3D dies, resulting in greater thermal accumulation within the hardware [2]. At the extreme scale, high heat can damage computing hardware and render it inoperable [3], creating a design limitation known as Dennard scaling [4]. To prevent such damage, thermal sensors in critical hotspots automatically trigger a decrease in the clock rate of devices to limit additional energy before the device becomes temporarily or permanently inoperable. To ensure performance and throughput, HPC and datacenter systems that push thermal limits require sophisticated cooling solutions. These solutions permit maximal usage for the longest possible period without detrimentally lowering clock rates or causing device failures.

Despite continued innovation in air-cooling technology, air is simply not dense enough to maintain pace dissipating heat from modern HPC and datacenter systems, let alone future designs that are predicted to only increase in Thermal Design Power (TDP). With nearly half of all datacenter energy devoted to cooling hardware and datacenters accounting for 2% of U.S. energy consumption [5], liquid-based cooling technologies are promising and necessary for continued growth

and innovation in computing. There are several different approaches to liquid-based cooling, including Direct Liquid Cooling (DLC) and Single- and Dual- Phase Liquid Immersion Cooling (SPLIC and DPLIC, respectively). We limit the focus of our discussion to SPLIC in this work.

### **2.1.1 SPLIC Technology**

SPLIC systems include a container with pumps for forced coolant circulation and tubes that can be connected to chilled water sources for heat exchange. Circulation is induced to reach temperature equilibrium of the entire fluid volume more quickly and to facilitate more effective heat exchange. In comparison to Direct Liquid Cooling (DLC) [6, 7], immersion cooling affects all computing components rather than just the areas covered by cold plates where liquid is always flowing. SPLIC systems are more energy-efficient because the pumps only activate periodically to circulate fluid or while the coolant temperature exceeds a set point.

## **2.2 The Word2Vec Algorithm and Its Optimization**

## **2.3 Performance Autotuning**

## Chapter 3

# Impacts of Immersion Cooling on Application Performance

### 3.1 Introduction

Many large-scale computer systems are transitioning or considering transitioning from air-cooling solutions to liquid-cooling solutions. Thermodynamic physics dictate that newer, more dense computing components will produce more thermal waste than air as a medium can physically circulate – our computers will melt down or have to operate in degraded performance simply because they’re too hot. As a denser physical medium, liquids can potentially improve over air in terms of cooling efficiency and costs while possibly also improving computing hardware reliability. Single Phase Liquid Immersion Cooling (SPLIC) is such a technology in which computing hardware is completely submerged in a container filled with a dielectric coolant fluid. Compared to air, the coolant medium is denser and has higher thermal conductivity and heat capacity. These properties improve heat dissipation and permit more efficient heat recycling. Many benefits of SPLIC are well-understood in theory, but the practical effects of the technology on real software workloads have not been well-studied academically.

Previous studies generally examine the raw heat circulation properties of the coolant without considerations for particular applications or hardware configurations. However, different hardware components possess different power requirements and ranges and patterns of heat dissipation based

on the software they are executing. For example, processing units consume more power and have a larger power range compared to memory devices and disk drives. Component power is also determined by the underlying hardware technologies; for instance, CPUs are generally less power efficient than GPUs, and DDR memory devices are less power efficient than high-bandwidth memory (HBM) devices for the same capacity. Finally, applications are the ultimate drivers of component utilization, meaning that different kinds of applications produce different cooling requirements from the same system. This means that the actual cooling demand of computing systems is highly variable across different hardware and software configurations.

Cooling’s role in compute systems is to prevent damage to computing hardware and to reduce or eliminate measures individual components utilize to prohibit damage, such as downclocking, which ultimately cause performance losses. While most software applications operate blissfully unaware of the thermal conditions of computing hardware, intense and long-running processes are well-known to lead to performance degradation and faults that reduce or deny access to peak computing potential. The real cooling demands required by a system for sustained performance are neither represented by idle conditions nor the maximum draw from running all components at full capacity at all times. Therefore a static understanding of peak heat dissipation capability does not paint a reasonable expectation of how cooling may affect system performance.

**TLR: This may be background rather than introduction** SPLIC has different cooling mechanisms, controls, and operational capabilities compared to air cooling. SPLIC dissipates heat in liquid coolant by circulating chilled water through tubes passing through the coolant tank. To actively exchange heat, the liquid coolant is moved through the heat exchange by using pumps on periodic schedules or when the coolant temperature exceeds a certain threshold. Unlike air cooling, which directly monitors computer hardware components and responds to their temperatures, SPLIC monitors the temperatures of the coolant and water entering and leaving the heat exchange within the tank. This approach to thermal monitoring is physically removed from the computing components, which inherently delays the system’s response to heat accumulation within computing hardware. Therefore, understanding the actual impacts of this cooling technology on different hardware and software workloads is important for evaluating its utility and design implications beyond mere hardware configuration.

In this chapter, we conduct a case study on thermal behaviors in a SPLIC tank with a immersed compute cluster. We utilize several CPU-based and GPU-based High Performance Comput-

ing (HPC) and datacenter applications with different compute and memory intensities and analyze hardware thermal behavior and their variations with the SPLIC system configurations and operations. We also compare the performance on highly similar hardware configurations in a traditional air-cooled datacenter rack. Unlike prior works that rely on simulations, our case study provides empirical insights on thermal demands induced by applications and their effects within the immersed environment.

## 3.2 Related Work

Many design decisions impact the performance of SPLIC, including the number of pumps, the pump arrangements and settings within the tank, and the choice of dielectric fluid, which have all been studied by many works [3, 8, 9], primarily via simulation or isolated experiments on individual components. Existing work has addressed several initial concerns about the technology, including the extent of dangers to hardware components from absorbing dielectric fluids [10] and the capital burden of procuring, installing and maintaining SPLIC rather than other more traditional cooling systems [11]. However, the existing literature largely lacks evaluations of SPLIC hardware in meeting actual application cooling demands, which we aim to address in this work.

TLR: For dissertation, I think a longer-form discussion of this could be merited. In particular, rather than collectively saying they discuss these things, we can back up the point in the introduction about what they *do analyze* and better establish the gap in research that we intend to address.

## 3.3 Research Questions

We investigate specific questions with real application performance in an SPLIC environment to provide a more comprehensive perspective on the larger subject.

### 3.3.1 RQ1: What are the thermal behaviors of individual hardware components in SPLIC?

Heat accumulates in the pod as power is drawn and used by individual computing components. The heat accumulated at a particular component generally increases as that component

utilizes more of its peak capability and when high utilization is sustained for extended durations. Most commodity hardware is technically compatible with SPLIC, but have thermal designs that do not explicitly consider properties of ambient liquid coolant. For instance, processors and accelerators typically comprises physical parts such as memory caches and processing cores accommodating both memory access and compute. Such integration improves performance but has a high thermal conductivity and heat exchange within components. SPLIC coolant, if not actively circulated all the time, is unable to reach thermal equilibrium or transfer heat as fast as fan cooling between components in close physical proximity.

It is important to understand if high core temperatures dissipate more heat to proximal components such as memory, particularly for GPUs. Furthermore, the environment’s tendency for equilibrium may result in certain hardware components operating at higher temperatures than would be typical given their power design, necessitating cooling interventions for components that may not have DVFS or thermal throttling capabilities.

We observe the actual thermal operating range and rate of temperature change for key hardware components before, during, and after prolonged execution of various applications. These characteristics can be compared relative to the range of temperatures and rate of temperature change in the tank’s coolant to determine an appropriate response and tolerance within the environment. We also observe the rate of change in hardware hotspot temperatures and coolant temperature to determine if the indirect thermal measurement performed by the cooling system can appropriately respond to high demand from the computing components. We present per-component analyses and coolant-relative per-component analyses in Section 3.5.1.

### **3.3.2 RQ2: How do initial conditions affect SPLIC performance and efficiency?**

Some liquid cooling techniques, such as DLC, yield higher heat dissipation rate as the coolant temperature rises, which is a very convenient upside. We aim to determine whether similar benefits can be observed in SPLIC by correlating heat dissipation with the initial temperature during cooling periods, ensuring that workloads do not interfere with the measurements. We also correlate various component activity measures with temperature dissipation during cooling cycles when applications are running to determine if different workloads exhibit distinct thermal patterns beyond the rate of

heat introduction into the environment. We analyze these trends in Section 3.5.2.

### **3.3.3 RQ3: How do different workloads affect heat accumulation?**

Exhaustive studies of application behavior are impractical. In this work, we use a selection of workloads to represent HPC and datacenter applications with similar primary resource demands and study the resulting thermal challenges for the cooling system.

In particular, we consider applications where the primary device used for computation is CPU or GPU and if the computation is subject to a bottleneck in compute or memory throughput. Due to the small number of servers, we exclude network throughput as a bottleneck in our study, though we recognize its importance for large-scale distributed system and workloads. We present a correlative analysis between workload classification and induced cooling demand in Section 3.5.3.

### **3.3.4 RQ4: What differences are observed compared to air-cooled environments?**

Finally, it is important to understand the experienced differences between air- and liquid-cooled environments. We focus on the differences in response times, rate of heat accumulation, and power utilization across experimental conditions in Section 3.5.4.

## **3.4 Experiment Design**

We answer the research questions by conducting a set of experiments and analyses as a representative case study. Our experimental platform consists of an SPLIC environment with immersed servers, a suite of applications with various thermal footprints, and tools to monitor sensors and performance.

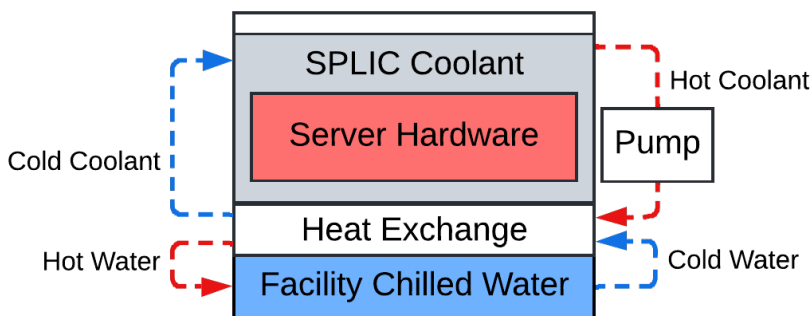
### **3.4.1 SPLIC and Server Hardware**

#### **SPLIC Environment**

Our SPLIC environment is visualized in Figure 3.1. We use a Submer SmartPod v3, which utilizes a proprietary synthetic dielectric fluid to cool all computing hardware within the tank’s volume. The tank (also referred to as a cooling pod or “pod”) uses two redundant pumps to circulate

coolant. The pod exchanges heat with facility-chilled water, which flows through tubes at the bottom. The temperature of the in-flow chilled water is measured to be inclusively between 10 to 12 degrees Celsius at all times.

We are limited to only a few servers available for immersion, which cannot generate enough heat in a short period for our studies if the pod is always connected to the chilled water. To address this limitation, we conduct thermal analyses while the chilled water is disconnected and continue after reconnecting chilled water to observe the system’s capability to respond to strong cooling demand. This approach allows us to study the SPLIC system’s response to high-heat environments without excessively stressing the system for prolonged periods. It also allows us to better understand the capacity of the coolant medium itself in our experiments, which could be overly-protected by the manufacturer’s minimum settings that require circulation every half hour for at least five minutes. Each of our experiments maintain high levels of compute activity on a single application without access to chilled water for an extended duration before halting the application and re-engaging the chilled water, permitting the system to return to its normal state.



**Figure 3.1:** SPLIC tank arrangement used in our experiments.

### Server Hardware

We immerse three server nodes into the pod. The nodes are composed of commodity hardware and colloquially referred to as “deepgreen,” “n01,” and “n02”, as detailed in Table 3.1. The deepgreen node is the head node directing all application flows while n01 and n02 are compute nodes that provide additional compute capability to scale application performance.

In this work, we are limited to commodity servers, which generalize across various possible



server configurations. We also leverage this to be as reasonably comparable to current air-cooled hardware configurations later in our work; our air-cooled hardware is detailed in Table 3.2.

**Table 3.1:** Immersed Server Hardware

| Nodes     | Kind         | Hardware                        | Specs                       |
|-----------|--------------|---------------------------------|-----------------------------|
| Deepgreen | Mother-board | S8021GM2NR-2T                   | Five (5) PCIe 3.0 x16 slots |
|           | CPU          | AMD EPYC 7551P                  | 2.00 GHz<br>32 cores        |
|           | GPU          | Two (2) NVIDIA Titan V          | 5120 cores<br>12 GB HBM2    |
| n01, n02  | Mother-board | X9DRG-QF                        | Four (4) PCIe 3.0 x16 slots |
|           | CPU          | Intel(R) Xeon(R) CPU E5-2670 v3 | 2.30 GHz<br>64 cores        |

**Table 3.2:** Air-Cooled Server Hardware

| Nodes        | Kind         | Hardware                    | Specs                       |
|--------------|--------------|-----------------------------|-----------------------------|
| Palmetto S12 | Mother-board | DELL 0D9WDC                 | Four (4) PCIe 3.0 x16 slots |
|              | CPU          | Intel(R) Xeon(R) E5-2680 v4 | 2.40 GHz<br>28 cores        |
|              | GPU          | Two (2) NVIDIA P100         | 3584 cores<br>12 GB HBM2    |

## Hardware Preparation for Immersion

The SPLIC coolant fluid includes plasticizer, so we replace thermal paste between hardware components and heat sinks with indium foil. The plasticizer also makes ethernet and power cables rigid and somewhat brittle after a few months. However, these cables remain at negligible risk of breaking while undisturbed.

We remove or disable all moving hardware on power supplies, motherboards, GPUs and CPUs. Server components designed to physically move in an air-cooling environment are unsuitable for immersion cooling. Take fans, for example; they would generate enormous heat from friction with the dense synthetic fluid, triggering alarms and risking burning<sup>1</sup>. Specialized hardware “immersion ready” or designed to improve SPLIC performance circumvents these issues.

---

<sup>1</sup>We indeed experienced this once.

### 3.4.2 Applications

We utilize a selection of HPC applications with different compute intensities, as shown in Table 3.3.

**Table 3.3:** Selected applications and classifications

| Application Name | GPU Accelerated | Throughput Bottleneck |
|------------------|-----------------|-----------------------|
| CUDA Stream      | ✓               | Memory                |
| EMOGI            | ✓               | Memory                |
| CUDA DGEMM       | ✓               | Compute               |
| MD5 Bruteforcer  | ✓               | Compute               |
| NPB DT Class=C   | X               | Memory                |
| NPB IS Class=D   | X               | Memory                |
| NPB EP Class=E   | X               | Compute               |
| HPCC HPL         | X               | Compute               |

#### GPU Applications

We select CUDA Stream as a GPU-enabled memory bandwidth test with minimal computation. For a more realistic representation of memory-intensive GPU-accelerated HPC applications, we utilize a manually constructed graph using the EMOGI graph tool [12]. This graph is designed to induce a memory-antagonistic BFS traversal when executed via EMOGI, permitting minimal data reuse while extending data lifetimes as long as possible per the algorithm’s implementation.

We also include compute-intensive GPU applications, including CUDA DGEMM, a common compute-intensive kernel utilized within many scientific and machine learning workloads. While machine-learning implementations may prefer lower precision datatypes than doubles (the “D” in DGEMM), we note that many scientific applications have not embraced lower precisions and that most of the computational instructions for lower precisions practically resolve into multiple words being loaded into the same double-precision pipeline for execution. We also provide the MD5 Bruteforcing algorithm as an example of datacenter workloads. This application’s repeated hash computations permit long arithmetic instruction sequences without dependence upon large-scale memory accesses or highly variable control flows.

## CPU Applications

We utilize several key kernels from the NAS Parallel Benchmark Suite [13], including Data Traffic, Integer Sort and Embarrassingly Parallel. Each of these benchmarks are executed at the largest class size that can be executed while utilizing the entire server system. As these classes are designed based on the number of available MPI ranks, we note that the air-cooled replication occasionally utilizes smaller classes than those executed via the immersed server.

For memory-bottlenecked CPU applications, we focus on Data Traffic and Integer Sort. The Data Traffic benchmark measures the communication performance between cores and over interconnects in the parallel environment, demonstrating the memory latency and bandwidth constraints on the system. The Integer Sort benchmark produces a large, in-memory list of integers that are sorted in parallel with coordination between processors.

For compute-bottlenecked CPU applications, we utilize the NPB Embarrassingly Parallel benchmark and HPCC HPL benchmark. The Embarrassingly Parallel benchmark application represents highly parallel compute saturation with minimal coordination between parallel components and interconnections. Similarly, the High Performance Computing Challenge (HPCC) High Performance Linpack (HPL) represents a solver for a random dense linear system in double precision arithmetic [14].

### 3.4.3 Metrics and Monitoring Tools

We utilize multiple publicly available software tools to collect metrics on CPU, GPU and NVMe device thermal sensors and activity measures, as well as a vendor-provided API to monitor the SPLIC system. We monitor the power of the SPLIC system over SNMP interfaces to the PDUs that all components draw power from, and similarly monitor power usage of the air-cooled server via vendor-provided reporting tools. To minimize the impact of monitoring on application performance, our tool is designed to sample each metric at a target rate of 1Hz and caches some information to reduce the overhead of sample collection. Our software harness is publicly available via GitHub at <https://github.com/tlrandal/LibSensorsTools>.

## CPU Monitoring

We record per-core frequencies from the `cpufreq/scaling_cur_freq` files in the Linux `/sys/devices/system/cpu/cpu*/` directories. These frequencies are recorded by the CPU governor in these files as integer-valued KHz.

We also use the `libsensors` library provided by `lm-sensors` (version 3.6.0) to monitor CPU core temperatures. The temperature values are reported as floating-point degrees Celsius with a single point of precision.

## GPU Monitoring

We record GPU metrics listed in Table 3.4 using the NVML library based on NVIDIA driver version 535.54.03 and NVML version 12020, corresponding to CUDA version 12.2. Each metric is represented using integer values, so temperature data are reported less precisely for GPUs than equivalent measurements for CPU temperatures.

**Table 3.4:** NVIDIA GPU metrics recorded using NVML

| Metric             | Meaning (Units)                             |
|--------------------|---------------------------------------------|
| GPU Temperature    | Average SM temperature (°C)                 |
| Memory Temperature | Memory junction temperature (°C)            |
| Power Usage        | Power draw (mW)                             |
| Power Limit        | Maximum power draw (mW)                     |
| GPU Utilization    | NVIDIA metric of SM activity (%)            |
| Memory Utilization | NVIDIA metric of memory activity (%)        |
| Memory Used        | Allocated global memory (bytes)             |
| Performance State  | NVIDIA metric of device state (integer 0-8) |

## NVMe Monitoring

We record NVMe temperatures using `libnvme` version 1.6, which are also reported as integer degrees Celsius.

## Power Monitoring

We record single-precision PDU phase amperage to determine the complete system’s power draw over an SNMP interface. The Rack PDUs for the immersed system are Schneider Electric

model AP7811B, with a 30 Amp limit and 208 Volt output of single-phase AC current, so the conversion to Watts is straightforward.

We also collect power consumption within the rack-level of the air-cooled server by the Dell OMReport tool, which also provides measurements with a single point of precision and directly calculates the instantaneous wattage.

## Pod Monitoring

We use the provided API endpoint to collect metrics listed in Table 3.5 using LibCurl version 7.68.0. The API reports temperatures at single-precision and most other values as integers.

**Table 3.5:** Submer SmartPod metrics recorded from vendor-provided API

| Metric               | Meaning (Units)                             |
|----------------------|---------------------------------------------|
| Temperature          | Average coolant temperature (°C)            |
| Consumption          | Pod power consumption (W)                   |
| Dissipation          | Thermal dissipation (both as °C and KW)     |
| mPUE                 | Power Usage Effectiveness (scalar)          |
| Pump RPM             | Pump rotations per minute (scalar)          |
| Coolant Temperatures | Input/output coolant temperature (°C)       |
| Water Temperatures   | Input/output chilled water temperature (°C) |
| Flow Rates           | Flow rate of coolant and water (L/minute)   |

### 3.4.4 Experimental Procedure

We conduct independent tests utilizing a single application as the workload for the server for the entire observed duration. Each test begins with thirty minutes of idling activity while disconnected from chilled water to establish a baseline for experimental conditions. After this initial period, we repeatedly execute the workload application for 7.5 hours, after which we reconnect the chilled water supply to the pod. We continue to monitor temperatures for an additional 24 hours as the pod dissipates accumulated heat and returns to its equilibrium state.

Due to potential harms to the air-cooled hardware, we do not disable any cooling components for our replicated experiments on these servers. We perform the same pre- and post- application idling periods to maintain as similar of conditions as possible between environments, but the cooling components are unaltered throughout our entire duration of monitoring. We also shorten the post-application idling period relative to the immersed server simply due to less time being required for

air-cooling to return the system to baseline conditions, which are shown in our results.

## 3.5 Results

We report our results based upon the research questions posed in Section 3.3. The first three research questions are exclusive to the immersed environment, with the fourth question also including data from the air-cooled environment.

### 3.5.1 Thermal Behaviors of Individual Hardware Components (RQ1)

**Importance.** The range of measured temperatures of each hardware component determines the risk of violating thermal tolerances and how quickly temperatures may be expected to change. We also seek to identify variations in temperature based upon the application demand to determine if thermal behavior is purely driven by energy expenditure or if more complicated behaviors emerge.

**Metrics.** We begin by analyzing the minimum temperature as a small range of values collected during the idle baseline prior to each application test. As initial conditions are similar across all experiments, we aggregate all data during these periods to establish the minimum, mean and maximum observed temperatures of each component.

We then consider the recorded temperatures across each experiment’s active application periods to determine the greatest attained temperature over the baseline and the greatest linear rate of temperature increase throughout all experiments. We compare the dissipated coolant heat relative to hardware activity measures to determine if more complex modeling is needed than pure power demand. The hardware activity measures and collection techniques are defined in Section 3.4.3.

**Results.** Each of the above metrics are computed for all monitored hardware and presented in Table 3.6. All hardware components except the NVMe are capable of changing temperatures faster than the pod coolant, however only the GPU hardware was observed to reach levels where thermal throttling could become a reasonable concern when executing the DGEMM application, where it reached a maximum temperature of 96 °C. The Titan V GPU models are designed for a maximum operating temperature of 91 °C, with thresholds for slowdown and shutdown at 97 and 100 °C, respectively. Had the experiment continued beyond the eight allotted hours, the disconnected chilled water would not permit adequate heat exchange for the SPLIC system to protect immersed hardware and could have permanently damaged the GPUs. This represents a somewhat reasonable

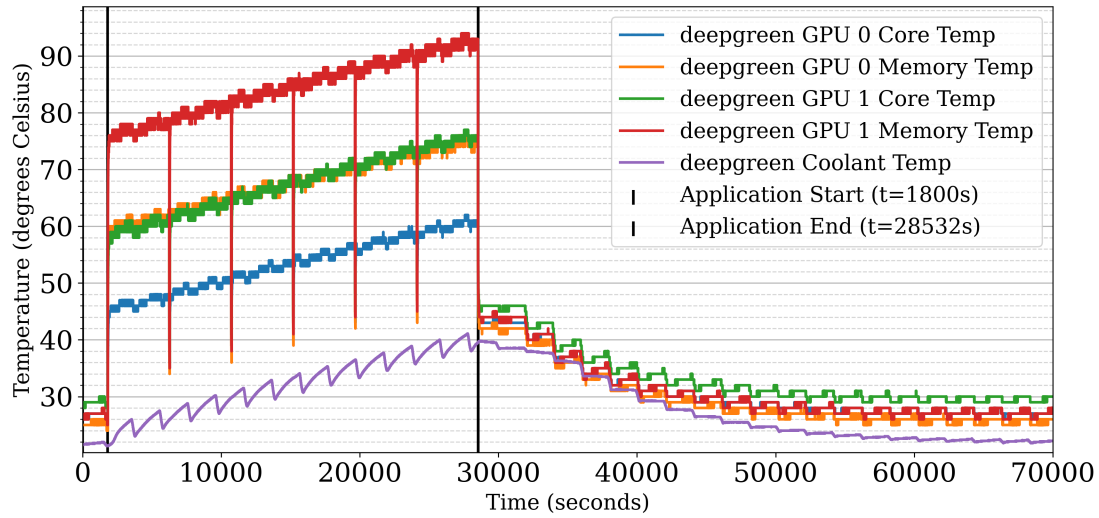
upper bound for our experimental conditions, however we note that our pod has additional coolant volume as not all compute racks are filled. A pod with all racks completely filled will both have more power draw within the volume and a lower ratio of coolant to heat-producing mass. This may require a faster response if chilled water becomes inaccessible to preserve the integrity of the system.

Notably, GPU temperatures fell to 46 °C almost immediately upon terminating the DGEMM application for this experiment, and a repeated execution of the benchmark without interrupted access to chilled water failed to exceed a GPU temperature of 80 °C. With the re-introduction of chilled water for heat exchange, the SPLIC system was able to return all temperatures to normal idling levels over the next 9 hours. Our results cannot generalize across all vendor products and configurations, but we believe this forms one of the most reliable stress-tests to calibrate emergency response thresholds. Figures 3.2 and 3.3 show detailed temperatures of this experiment in without chilled water and with uninterrupted access to chilled water.

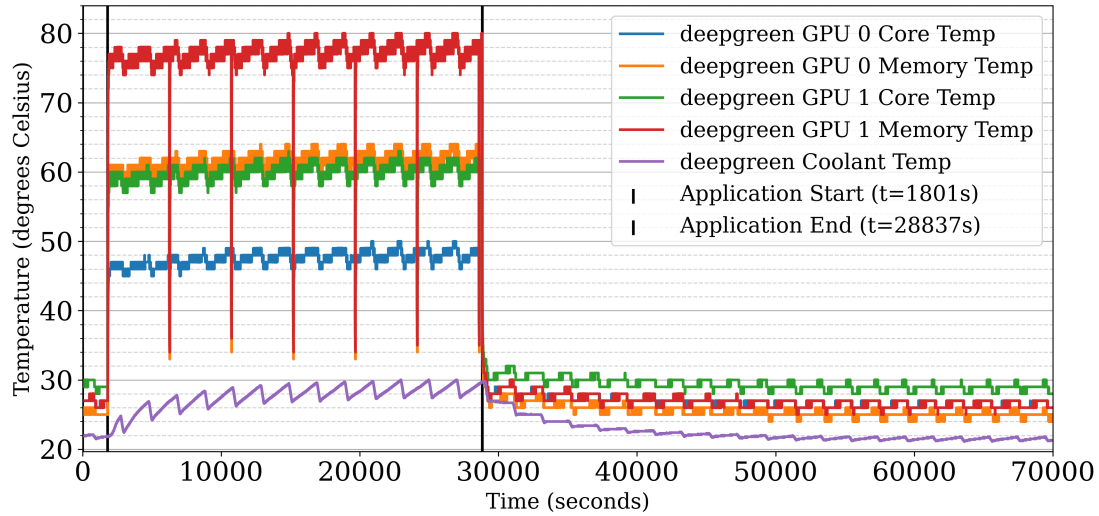
TLR: Rong noted that the memory temperatures are higher, which is typically not the expectation – this is odd because currently the core is idling higher than the memory temperature, are our labels incorrect? If so, relabel the figures and see if the LibSensors labels are just recorded backwards vs collection

**Table 3.6:** Thermal Behaviors by Hardware Component

| Hardware Component | Idle Temperature Minimum/Mean /Maximum (°C) | Interval (s) to Increase Temp by 1°C | Maximum Observed Operating Temperature (°C) |
|--------------------|---------------------------------------------|--------------------------------------|---------------------------------------------|
| SPLIC Coolant      | 19.80 / 21.41 / 22.50                       | 970.99                               | 47.10                                       |
| deepgreen CPU      | 14.75 / 17.51 / 19.66                       | 702.20                               | 55.62                                       |
| deepgreen GPU      | 24.25 / 26.54 / 28.00                       | 266.83                               | 96.00                                       |
| deepgreen NVMe     | 14.00 / 17.91 / 20.00                       | 1068.48                              | 44.00                                       |
| n01 CPU            | 17.44 / 21.30 / 24.04                       | 607.33                               | 66.00                                       |
| n02 CPU            | 17.67 / 20.81 / 24.26                       | 560.35                               | 69.00                                       |



**Figure 3.2:** Thermal Behavior of DGEMM Without Chilled Water. DGEMM runs on GPU 1.



**Figure 3.3:** Thermal Behavior of DGEMM With Chilled Water. DGEMM runs on GPU 1.



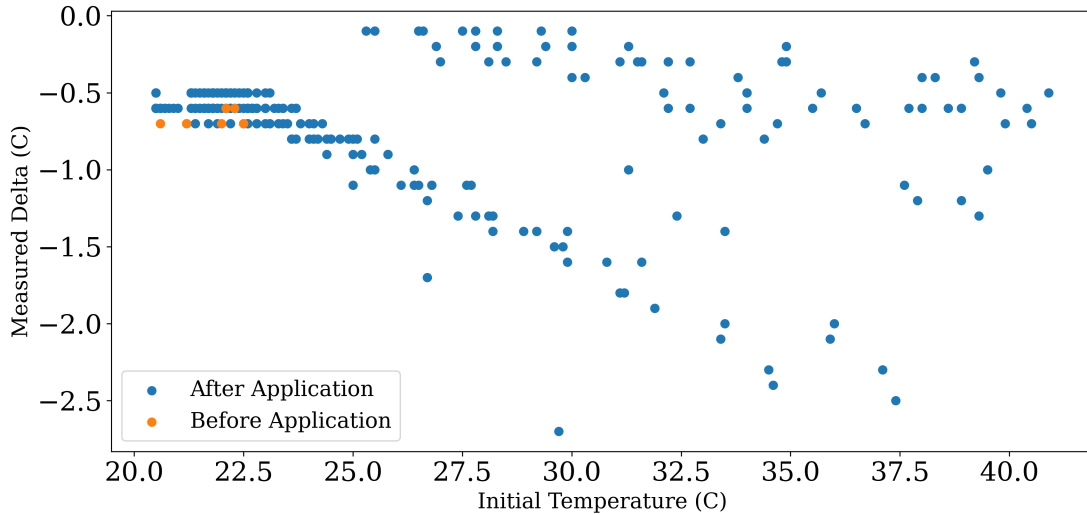
### 3.5.2 Effects of Initial Conditions (RQ2)

**Importance.** SPLIC’s energy efficiency may correlate with fluid temperature, making a trade-off between maximum thermal tolerance and long-term energy expenditure.

**Metrics.** We monitor the dissipated heat during periods where pumps are active and chilled water is available for the SPLIC heat exchange.

**Results.** We present the aggregated data across all observations in Figure 3.4. It’s clear that the maximum heat removed positively correlates with a higher initial temperature, however the minimum heat removed during a cycle remains roughly constant regardless of initial temperature. Because the pump activity duration, chilled water temperature and pump power draw are held invariant in our experiments, this implies that SPLIC can yield increased energy efficiency at higher temperatures, however the behavior is not guaranteed to be observed.

TLR: If these are re-labeled by applications, do certain classes of application do better at following the linear trend? Because that changes where the heat is being removed from.



**Figure 3.4:** Temperature at start of idle pump cycle vs amount of heat removed.

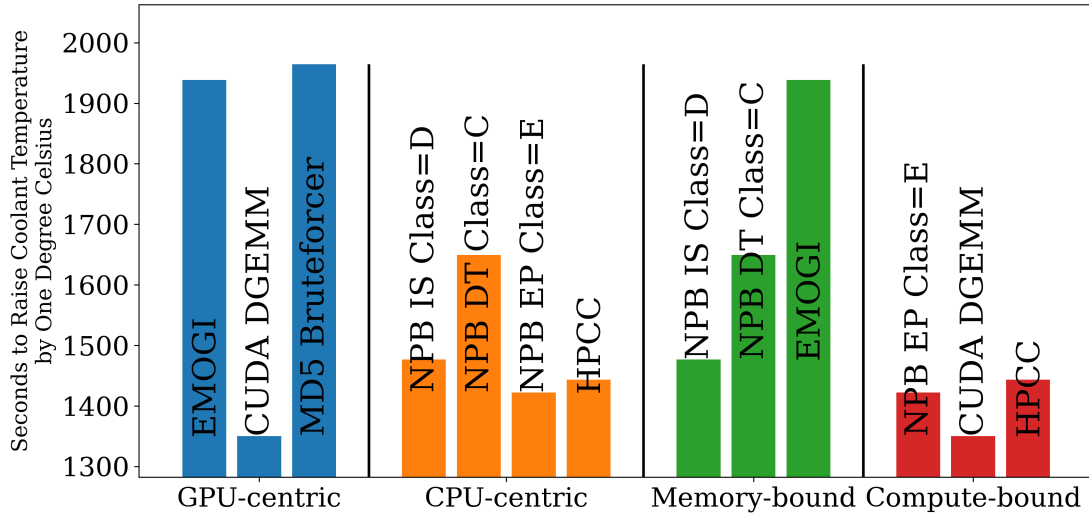
### 3.5.3 Effects of Workloads on Heat Accumulation (RQ3)

**Importance.** Given that almost all hardware components can heat up faster than the average coolant temperature, it’s important to know what workloads will likely produce the greatest

heat accumulation within the pod. This simplifies benchmarking a system’s cooling demands and ensures system responses can be properly calibrated for extended application executions.

**Metrics.** We use the average interval to increase the temperature by one degree Celsius when the temperature delta reaches the maximum during application execution. To simplify comparisons, we group applications based upon CPU- or GPU-centric classifications and application throughput being Compute- or Memory-bound as previously denoted in Table 3.3.

**Results.** We display the interval of each experiment application in Figure 3.5. The lower values indicate faster heat accumulation. These applications present a good range of heat accumulation rates. In contrast to common impression that GPU applications accumulate heat faster, only compute-intensive GPU applications do, while memory-bound GPU applications yield relatively slow temperature changes. In general, compute-bound applications accumulate heat faster than memory bound applications, no matter whether they are programmed to run on CPU or GPU.



**Figure 3.5:** Time intervals required by applications to accumulate heat. The smaller the interval, the faster the application execution increases coolant temperature by 1 °Celsius.

### 3.5.4 Comparison to Air-Cooled Environment (RQ4)

TLR: Add figures and analysis here similar to other sections

## 3.6 Conclusions

TLR: Update all of this to include air-cooling conclusions as well

Our experiments reveal a few initial insights about the current state of SPLIC immersion technology.

**Strong resilience to thermal changes.** Current tank designs use temperature readings at coolant pumps, instead of relying on embedded sensors in hotspots and computer components, to respond to thermal changes. This is a fixed strategy. As evidenced by our experiments, the technology can handle fluctuations in computing demand for extended durations. However, the same physical properties that enhance its energy efficiency also significantly extend the time required to return to a neutral state.

**Delayed responses to temperature differentials.** Our study shows that hot spots within hardware components have significant lag time before pod sensors can detect changes in temperature, meaning that responses must be calibrated to be more aggressive in inducing a cooling response in case hardware conditions outpace the system’s detection. Future iterations of the hardware that can directly read computer components’ sensors can respond in a more appropriate fashion without expending extra energy.

**Ease of operation and maintenance.** Modifying commodity hardware for initial server setup is intrusive but can be circumvented by utilizing specialty hardware designed for SPLIC. Non-specialized hardware can be limited by server layout, such as connectors that cannot bend sharply to reach ports and certain components that are inaccessible for maintenance without completely removing server blades due to the pod rack’s vertical orientation. After removing hardware from the pod for maintenance, liquid residue can create slipping hazards and must be carefully monitored and cleaned. Finally, adding or removing components in the pod changes the level of coolant, which may require adding or removing coolant to maintain an appropriate volume for the container.

**Reliability.** Throughout our extended immersion of hardware, we have not observed degradation in peak performance. Plastic components affected by the plasticizer are eventually damaged and may require replacement during or after maintenance.

After our experiments, our datacenter had a power outage that we believe caused damage to a coolant pump sensor. This left the pod in a malfunctioning state that caused the coolant pumps to fail, leading us to terminate future experiments until a replacement could be sourced and

installed. This level of operation disruption was not easily addressed by our technicians without vendor involvement, which complicates service and maintenance.

## Chapter 4

# PROPOSAL DEEPGREEN

**Does Improved Heat Dissipation Permit Sustained Performance** Due to the under-provisioning of compute hardware relative to the calculated potential heat dissipation of the Submer Pod, we temporarily disconnect the water for heat exchange to build heat in the system, then reconnect to observe the effects of passive and active heat dissipation within the system. We expect SPLIC to permit hardware to run for longer durations without reaching critical thermal levels that require throttling, leading to greater sustained application throughput. **Then, we will replicate these experiments using similar hardware configurations in an air-cooled environment for comparison, paying special attention to hotspot management.** We hypothesize that the air-cooled environment will require more energy to less effectively remove heat from the system, however as a more directed, active cooling system we expect it will return the entire system to its idle condition much faster than the SPLIC system.

**Specific Impacts on Different Hardware Components** Our experimental setup monitors memory, CPU and GPU components as well as the general environment. **We expect that greater thermal insulation provided by air will allow under-utilized components to remain cool, while the liquid coolant will freely share heat between hardware components.** It is possible that an abundance of dissipated heat could induce new thermal bottlenecks in other components.

## Chapter 5

# Summary Discussion and Future Work

TLR: Discussion has learnings/observations and the future outlook; then conclusion

**THIS SECTION HAS NOT BEEN UPDATED SINCE THE PROPOSAL**

This proposal seeks to address the necessity of collaboration between computer hardware and software for effective performance, especially in HPC environments. We demonstrate the opportunities present for hardware to better benefit software by analyzing novel SPLIC technologies and how they affect software performance relative to familiar air-cooled environments. We provide an example of modifying software to better adhere to hardware accelerator design by optimizing the GPU performance of Word2Vec based on core principles of hardware implementation and strengths, permitting both retroactive and proactive performance boosts without need for code modification specific to newer hardware generations. Finally, we demonstrate how the presence of tunable hardware and software requires special care to properly configure both components for maximum performance. To increase the efficiency of repeated optimization on similar tuning spaces, we introduce a novel performance autotuning technique that requires minimal data to produce insightful suggestions for attaining high performance within a small number of empirical trials.

## 5.1 Research Questions to be Answered

**Impact of Immersion Cooling on Application Performance** Our comparative studies of application performance between air-cooled and SPLIC environments will demonstrate the impact of cooling not just on energy efficiency, but on application performance. We anticipate our research on SPLIC systems will demonstrate that SPLIC is more energy efficient and has greater passive and active heat dissipation than air-cooling, but is less responsive to hotspots in the system and can actually be slower to address thermal issues in the system when heat accumulates. We expect that applications which thoroughly saturate available resources, especially power-hungry resources such as GPU compute, may continue to experience thermal throttling and may even induce thermal throttling in other components due to heat diffusion properties of the coolant fluid.

### **Properly Leveraging GPU Acceleration: A Case Study in Word2Vec Optimization**

Prior implementations of the GPU algorithm for Word2Vec experienced insufficient performance scaling due to hardware bottlenecks that were poorly handled by the software. Easing these bottlenecks can permit proper performance improvements from successive generations of GPU hardware even without code modification specific to particular architectures, improving the general benefit from acceleration with limited cost of development. While not all applications suffer from the same issues that Word2Vec experienced, nor do all application developers have the resources to rethink implementation for different accelerator architectures, our generation-agnostic optimizations demonstrate the effectiveness of general performance enhancements that can be tuned to successive hardware generations.

**Efficient and Transferable Multi-Scale Performance Autotuning** Maximizing performance is not simple even when both hardware and software perspectives are properly accounted for. Different scales of data and hardware availability can dramatically alter performance constraints even on a fixed system, much less on a similar architecture that experiences a partial upgrade. Vast search spaces that are infeasible to tune by hand and the nontrivial variety of performance needs across different scales necessitate automatic tools that can efficiently leverage existing knowledge in new contexts for rapid performance tuning.

We have demonstrated the GC’s capability to rapidly prototype promising configurations based on input data scaling with minimal training data, but are still improving the technique to

handle weak-scaling performance tuning where both input data scale and hardware resources are tuned simultaneously. We anticipate our contributions will greatly reduce the overhead of continuous performance tuning and permit additional tuning at additional data and system scales. By reducing the cost of these tuning efforts, users of all kinds of applications can benefit from improved performance over the lifetime of their work and the system.

## 5.2 Recommendations for Further Research

We anticipate multiple future directions for research beyond this proposal.

Our work on SPLIC environments can be studied in a variety of other environments, such as those equipped with forced induction components that have localized pumps to increase coolant flow rate around critical hot spots. There are also many varieties of server designs for SPLIC including different pump configurations, coolants and container form factors that may be compared with our own results to indicate better ways to utilize SPLIC technologies. Other accelerators have different hotspots and thermal constraints, which can also be studied in SPLIC environments to determine their suitability and innovations in SPLIC technology that may better support the massive varieties of accelerator hardware.

While Word2Vec has received less attention in the wake of LLMs and transformer architectures, further improvements to our GPU implementation such as multi-GPU and multi-node scaling are possible. Additionally, we expect many latency-bound GPU applications can benefit from algorithm-specific modifications similar to our register and shared memory utilization that pivot away from common matrix-operation style computation towards vector-operation computation that the hardware may more efficiently process.

Finally, we look forward to other non-regressive transfer learning approaches for performance autotuning to further explore the regime of data-constrained few-shot transfer learning. We also expect that modified systems and applications can benefit from similar transfer tuning approaches, permitting the knowledge gained from tuning previous iterations to be reused for even broader and longer-lived benefit to system users.



# Bibliography

- [1] John L. Hennessy and David A. Patterson. A new golden age for computer architecture. *Commun. ACM*, 62(2):48–60, Jan 2019.
- [2] Venkat Natarajan, Anand Deshpande, Sudarshan Solanki, and Arun Chandrasekhar. Thermal and power challenges in high performance computing systems. *Japanese Journal of Applied Physics*, 48(5S2):05EA01, May 2009.
- [3] Electronic and Photonic Packaging Division. *Computational Analysis for Thermal Optimization of Server for Single Phase Immersion Cooling*, volume ASME 2019 International Technical Conference and Exhibition on Packaging and Integration of Electronic and Photonic Microsystems of *International Electronic Packaging Technical Conference and Exhibition*, 10 2019.
- [4] R.H. Dennard, F.H. Gaensslen, Hwa-Nien Yu, V.L. Rideout, E. Bassous, and A.R. LeBlanc. Design of ion-implanted mosfet’s with very small physical dimensions. *IEEE Journal of Solid-State Circuits*, 9(5):256–268, 1974.
- [5] Richard Brown, Alliance to Save Energy, ICF Incorporated, ERG Incorporated, and U.S. Environmental Protection Agency. Report to congress on server and data center energy efficiency: Public law 109-431. Technical report, Lawrence Berkeley National Laboratory, 2008. Retrieved from <https://escholarship.org/uc/item/74g2r0vg>.
- [6] Bharath Ramakrishnan, Yaser Hadad, Sami Alkharabsheh, Paul R. Chiarot, and Bahgat Sammakia. Thermal Analysis of Cold Plate for Direct Liquid Cooling of High Performance Servers. *Journal of Electronic Packaging*, 141(4):041005, 07 2019.
- [7] A. Bar-Cohen, M. Arik, and M. Ohadi. Direct liquid cooling of high flux micro and nano electronic components. *Proceedings of the IEEE*, 94(8):1549–1570, 2006.
- [8] Shengchun Liu, Zhiming Xu, Zhiming Wang, Xueqiang Li, Haiwang Sun, Xinyu Zhang, and Haoran Zhang. Optimization and comprehensive evaluation of liquid cooling tank for single-phase immersion cooling data center. *Applied Thermal Engineering*, 245:122864, 2024.
- [9] Harshad Shrigondekar, Yueh-Cheng Lin, and Chi-Chuan Wang. Investigations on performance of single-phase immersion cooling system. *International Journal of Heat and Mass Transfer*, 206:123961, 2023.
- [10] Jimil M. Shah, Keerthivasan Padmanaban, Hrishabh Singh, Surya Duraisamy Asokan, Satyam Saini, and Dereje Agonafer. Evaluating the Reliability of Passive Server Components for Single-Phase Immersion Cooling. *Journal of Electronic Packaging*, 144(2):021109, 10 2021.
- [11] Nugroho Agung Pambudi, Alfian Sarifudin, Ridho Alfian Firdaus, Desita Kamila Ulfa, Indra Mammad Gandidi, and Rahmat Romadhon. The immersion cooling technology: Current and future development in energy saving. *Alexandria Engineering Journal*, 61(12):9509–9527, 2022.

- [12] Seungwon Min, Vikram Sharma Mailthody, Zaid Qureshi, Jinjun Xiong, Eiman Ebrahimi, and Wen-Mei W. Hwu. EMOGI: efficient memory-access for out-of-memory graph-traversal in gpus. *CoRR*, abs/2006.06890, 2020.
- [13] David H. Bailey. *NAS Parallel Benchmarks*, pages 1254–1259. Springer US, Boston, MA, 2011.
- [14] Antoine Petitet, Clint Whaley, Jack Dongarra, Andy Cleary, and Piotr Luszczek. Hpl - a portable implementation of the high-performance linpack benchmark for distributed-memory computers. <https://www.netlib.org/benchmark/hpl/index.html>, 2018. Accessed: 2023-10-01.